

MPL Work Report

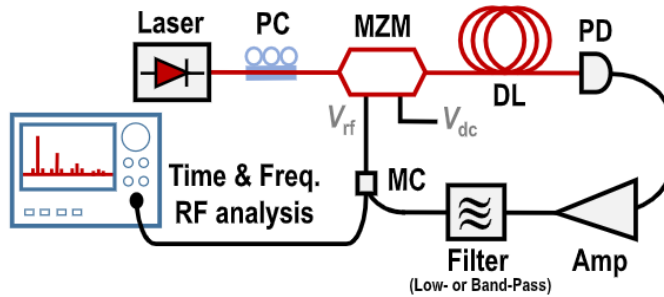
Cyrus Young, May - August, 2025.

Birgit Stiller Lab, Max Planck Institute of Light, Erlangen-Germany

1) Introduction to OEOs:

1.1) Theory:

Imagine we have a Optoelectronic Oscillator (OEO) that is an Ikeda-like system shown below:



Imagine we want to measure the intensity of the optical beam at the point below the MZM (Mach-Zehnder Modulator) and the MC (Microwave coupler). Let's call this point A. Because we are in an oscillator circuit, the intensity of the laser at that point will likely change over time. It would be nice to know how the intensity at that point would change over time. We do this by representing each block/component as a mathematical function.

At point A, we have a signal with intensity $x(t)$. It goes through a MZM which applies a nonlinear function to it, so now after the MZM, we have $f_{NL}(x(t))$. Next, it goes through a delay loop and thus undergoes a time delay so our original signal is now $f_{NL}(x(t - \tau))$. It then goes through a photodetector (which is a measurement device and doesn't affect the signal) and then an amplifier and thus the signal after the amplifier is $\beta f_{NL}(x(t - \tau))$. Lastly, the signal goes through a filter (called h), and now we are back at our starting point, point A (it should be noted that the Microwave Coupler does not have an effect on the device). Therefore, we can represent our system via the following equations:

$$x(t) = h * \beta f_{NL}(x(t - \tau))$$

$$h^{-1} * x(t) = \beta f_{NL}(x(t - \tau))$$

If we set $h^{-1} = \hat{H}$, we get the following equation:

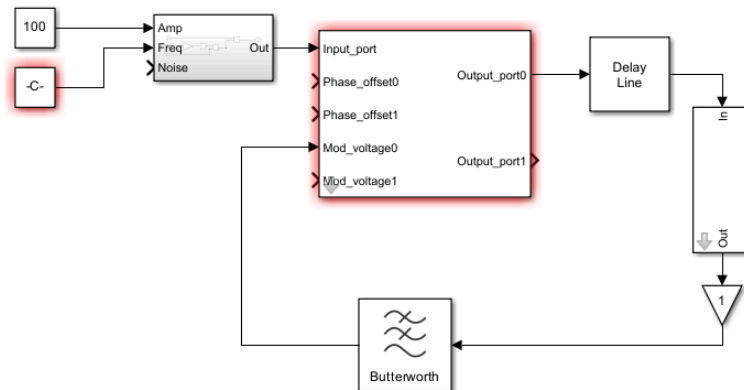
$$\hat{H}[x(t)] = \beta f_{NL}(x(t - \tau))$$

And in many cases, $\hat{H}[x(t)]$ is a differential equation with $x(t)$. This is important as it allows us to give us a mathematical model of how the output (usually the voltage or intensity of the signal) at a given point varies over time.

1.2) Option 1: Using Simulink to simulate OEO:

The first option is to use Simulink to simulate the behaviour of the OEO as a function of time. This method, as it turns out, is in many ways inferior compared to the next method (labeled Option 2) due to its lack of transparency and lack of versatility. It lacks transparency as the functions are hidden behind blocks - making it harder to detect errors - and lacks versatility as there is not a block for every component and many blocks have a limited set of inputs. In any case, this method is mentioned to show an initial approach.

Below is a Simulink diagram of our original OEO. Although the necessary components are present, making sure that the combination of components acts realistically is a task on its own and would be much easier to verify with our next option.



1.3) Option 2: MATLAB Code to Simulate OEO:

As opposed to using a Simulink simulation to model the system, it would be preferable to represent the system entirely via code. This will allow our representation to not only be more versatile (compared to the simulation which could only simulate a limited number of components) since we can now write code for each component, but also more transparent (as we are now writing the functions directly rather than using blocks). Below is the initial code to simulate an OEO as shown in [section 1.1](#). This code will undergo changes as time progresses to take into account optimization of the program and varying parameters:

```
clear; rng default % deterministic noise seed

%% Core Parameters:

fc = 10e9; % RF freq [Hz]
tauD = 20e-6; % fibre delay 20  $\mu$ s
Qrf = 3000; % RF band-pass Q
Glin = 35; % small-signal loop gain [dB]
Vpi = 4; % MZM half-wave voltage [V]
phiBias = pi/2; % quadrature bias for max linearity
Fs = 2e9; dt = 1/Fs; % envelope-level sample rate / step
Nt = round(1e-3/dt); % simulate 1 ms (>50 $\times$   $\tau_D$ )
Ndelay = round(tauD/dt); % integer delay in samples
assert(Ndelay>10, 'Increase Fs or shorten  $\tau_D$ ');

%% Blocks:

% MZM intensity transfer, bias  $\phi_b$ 
mzm = @(v) cos(pi*v/(2*Vpi) + phiBias).^2;

% Soft-limiting RF amplifier (unit saturation, Glin dB small-signal)
A0 = 10^(Glin/20);
amp = @(x) (A0*x)./max(A0,abs(x)); % gentle bend into 1 V max

% Base-band equivalent of RF band-pass (2-pole Butterworth LPF)
BW = fc/Qrf; f3 = BW/2; wn = 2*pi*f3; % rad/s cutoff
[b,a] = butter(2,wn/(pi*Fs)); % digital LPF

%% Run -----

bufOpt = zeros(Ndelay,1); % optical-delay circular buffer
y = complex(zeros(Nt,1)); % RF output record
noise = (randn(Nt,1)+1i*randn(Nt,1))*1e-6;

% IIR filter state for per-sample processing
zi = zeros(max(numel(a),numel(b))-1,1);
v_prev = 0; % previous RF sample feeding the MZM

for n = 1:Nt
```

```

% MZM (intensity transfer) -----
v_mzm = mzm(real(v_prev));           % intensity  $\propto \cos^2(\dots)$ 

% Fibre delay -----
k      = mod(n-1,Ndelay) + 1;        % circular-buffer index
v_del  = bufOpt(k);                  % intensity after  $\tau_D$ 
bufOpt(k) = v_mzm;                   % write current intensity

% Photo-detector (linear) -----
v_pd = v_del;                        % voltage  $\propto$  delayed intensity

% RF amplifier -----
v_amp = amp(v_pd);                   % soft-limiting gain

% Narrowband RF filter -----
[v_filt, zi] = filter(b,a,v_amp,zi); % 2-pole Butterworth

% Close the loop -----
y(n)  = v_filt + noise(n);           % record & seed with noise
v_prev = y(n);                       % feeds next-step MZM
end

%% Visualise -----

t = (0:Nt-1)*dt;
Y = fftshift(fft(y));
f = Fs*(-Nt/2:Nt/2-1)/Nt;
figure;
subplot(2,1,1), plot(t*1e3, real(y));
xlabel('Time (ms)'), ylabel('v_{RF} (a.u.)'), title('OEO start-up & steady-state');
subplot(2,1,2), plot(f/1e6, 20*log10(abs(Y)/max(abs(Y))));
xlim([fc/1e6-100 fc/1e6+100]), grid on
xlabel('Frequency (MHz)'), ylabel('Magnitude (dB)'), title('Spectrum near 10 GHz');

```

We will now delve into a detailed explanation of the actual functioning of the code. The first step is to define the core parameters of our system. f_c represents our oscillation frequency (i.e. the carrier frequency of which the system is defined). τ_D represents the loop delay. Q_{rf} represents the Quality factor. G_{lin} represents the small-signal loop gain - we use a small signal as this describes the gain before saturation. V_{pi} is the Mach-Zehnder Modulators half-wave voltage - the voltage swing needed to go from maximum to minimum optical output. ϕ_{Bias} represents the DC voltage bias at the MZM. F_s is the envelope-sampling rate. N_t represents the number of time steps needed to represent a certain amount of time in operation (in this case, the number of time steps to simulate 1 ms of operation - which lets you see around 50 round trips). N_{delay} represents the number of samples in the delay line. For both N_t and N_{delay} , we round the values to ensure that they are implemented in integer number of steps to ensure precision.

```

fc      = 10e9;                      % RF freq [Hz]

```

```

tauD = 20e-6;           % fibre delay 20  $\mu$ s
Qrf = 3000;             % RF band-pass Q
Glin = 35;              % small-signal loop gain [dB]
Vpi = 4;                % MZM half-wave voltage [V]
phiBias = pi/2;         % quadrature bias for max linearity
Fs = 2e9; dt = 1/Fs;    % envelope-level sample rate / step
Nt = round(1e-3/dt);    % simulate 1 ms (>50 $\times$   $\tau_D$ )
Ndelay = round(tauD/dt); % integer delay in samples

```

The next step is to make sure that τ_d , which represents the physical fibre time delay, is sufficiently represented in the program in order for the delay to be non-negligible.

```

assert(Ndelay>10, 'Increase Fs or shorten  $\tau_D$ ');

```

Our next operation is performing the corresponding MZM (Mach-Zehnder-Modulator) function on it. Here, the MZM applies a nonlinear effect on the signal, v , by applying a $\cos^2$ plus phase operation on it.

```

% MZM intensity transfer, bias  $\phi_b$ 
mzm = @(v) cos(pi*v/(2*Vpi) + phiBias).^2;

```

We then need to make sure the signal is amplified, and in a realistic fashion (i.e. we want it to eventually converge to some value, A_0 , as our signal becomes larger (thus mimicking the behavior of actual amplifiers). We do this by giving the following function, $amp(x)$, where x is the signal:

```

% Soft-limiting RF amplifier (unit saturation, Glin dB small-signal)
A0 = 10^(Glin/20);
amp = @(x) (A0*x)./max(A0,abs(x)); % gentle bend into 1 V max

```

We then implement a band-pass filter like so. What the following code below implements is a lowpass filter with a frequency band ranging from $0 \leq f \leq f_c / 2Q_{rf}$. The reason it is referred to as “base-band” is that physically, it would be equivalent to a bandpass starting just above 0 Hz, so in MATLAB we implement a low pass filter.

```

% Base-band equivalent of RF band-pass (2-pole Butterworth LPF)
BW = fc/Qrf; f3 = BW/2; wn = 2*pi*f3; % rad/s cutoff
[b,a] = butter(2,wn/(pi*Fs)); % digital LPF

```

The next step is to create the optical-delay circuit buffer. The first line, `bufOpt = zeros(Ndelay,1);`, allocates a column vector of length `Ndelay` to serve as a circular buffer for the optical intensity exiting the MZM. The code overwrites the oldest sample with the newest and reads from that same position rather than shifting an array at every time step - emulating the fixed fiber delay τD . Next, `y = complex(zeros(Nt,1));` pre allocates a complex-valued column vector of length `Nt` to record the RF loop output at each envelope sample. The reason it is complex from the outset is because this ensures versatility in the program by allowing both real and imaginary components in the output. Finally, `noise = (randn(Nt,1) + 1i*randn(Nt,1)) * 1e-6;` generates two independent Gaussian random sequences, combines them into a small-amplitude complex noise vector (on the order of 10^{-6}), and seeds each step of the loop with a perturbation. This tiny noise term breaks the symmetry of the zero-signal state and kick-starts the oscillation, while preallocating all arrays up front maximizes performance of the program.

```
%% Run -----
bufOpt = zeros(Ndelay,1);           % optical-delay circular buffer
y       = complex(zeros(Nt,1));     % RF output record
noise   = (randn(Nt,1)+1i*randn(Nt,1))*1e-6;
```

The first line allocates the initial internal state for the digital IIR filter. The second line sets the starting RF envelope voltage feeding into the MZM to 0. This is to ensure that the modulator begins from its unbiased quadrature point and that the filter state and ebay buffier are initially 0.

```
% IIR filter state for per-sample processing
zi = zeros(max(numel(a),numel(b))-1,1);

v_prev = 0; % previous RF sample feeding the MZM
```

The loop (which runs for each time step) shows how the optical signal evolves as it goes through each component. It first passes through an MZM - in which a nonlinear function is applied to it. It then passes through the delay loop - the current buffer index k is computed, the intensity after the delay is computed and the original slot for the buffer output is replaced by the new value.

The next step is to represent the photo-detector. In reality, this simply measures the voltage of the signal at that point - thus the code represents this by simply setting the voltage of the photodetector, v_pd , equal to that of the voltage exiting the delay loop. The signal then goes through an amplifier, in which the $amp()$ function defined previously in the program is applied.

The newly amplified signal then goes through the 2-pole Butterworth band-pass (implemented as a low-pass on the baseband envelope) while updating the internal state z_i . This selects frequencies near f_c and suppresses others. Lastly, some noise is added to our newly filtered signal and the loop is finished. The final value is then used as the initial value for the next loop, indicating the iterative nature of the program.

```

for n = 1:Nt
    % MZM (intensity transfer) -----
    v_mzm = mzm(real(v_prev));          % intensity  $\propto \cos^2(\dots)$ 

    % Fibre delay -----
    k      = mod(n-1,Ndelay) + 1;      % circular-buffer index
    v_del  = bufOpt(k);                 % intensity after  $\tau_D$ 
    bufOpt(k) = v_mzm;                  % write current intensity

    % Photo-detector (linear) -----
    v_pd = v_del;                       % voltage  $\propto$  delayed intensity

    % RF amplifier -----
    v_amp = amp(v_pd);                  % soft-limiting gain

    % Narrowband RF filter -----
    [v_filt, zi] = filter(b,a,v_amp,zi); % 2-pole Butterworth

    % Close the loop -----
    y(n)  = v_filt + noise(n);          % record & seed with noise
    v_prev = y(n);                     % feeds next-step MZM
end

```

Lastly, some final code is needed to visualize the output. The first graph shows the voltage output of the optical signal as a function of time, while the second graph shows the normalized RF spectrum near the carrier frequency of 10 GHz.

```
%% Visualise -----
t = (0:Nt-1)*dt;
Y = fftshift(fft(y));
f = Fs*(-Nt/2:Nt/2-1)/Nt;

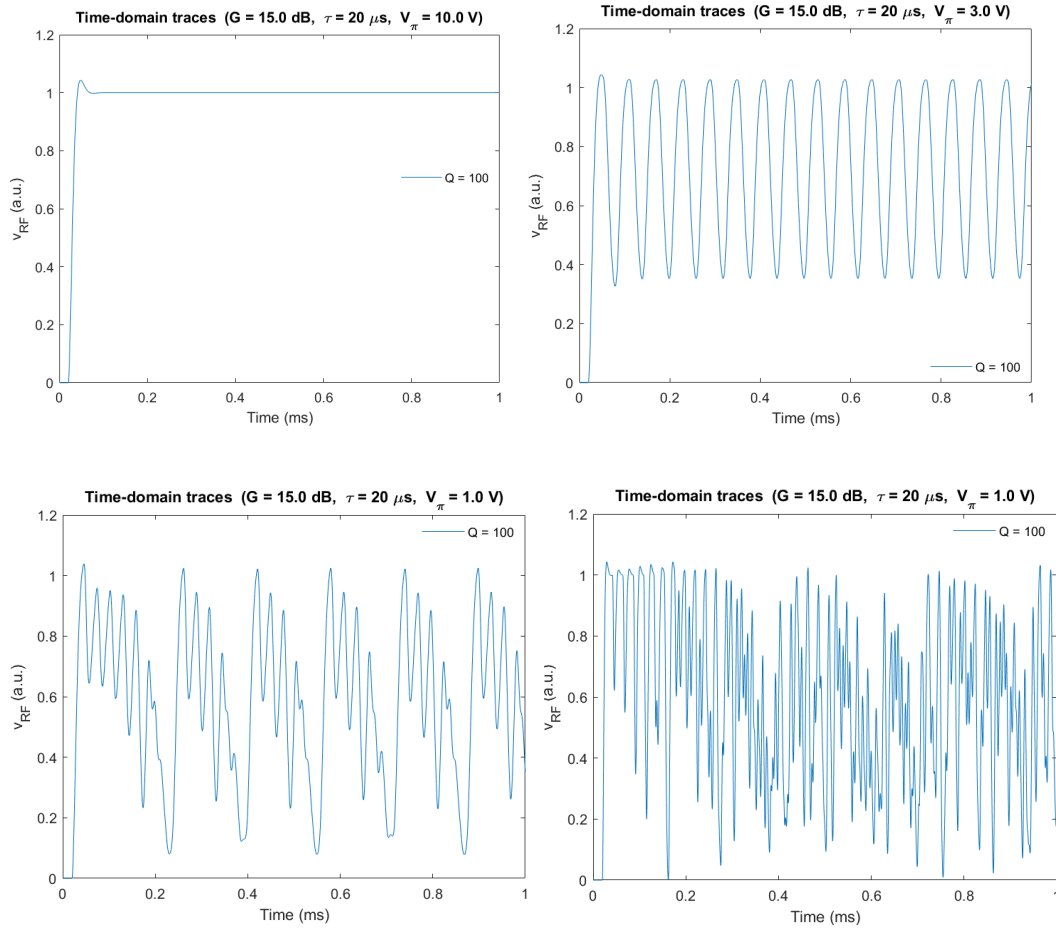
figure;
subplot(2,1,1), plot(t*1e3, real(y));
xlabel('Time (ms)'), ylabel('v_{RF} (a.u.)'), title('OEO start-up & steady-state');
subplot(2,1,2), plot(f/1e6, 20*log10(abs(Y)/max(abs(Y))));
xlim([fc/1e6-100 fc/1e6+100]), grid on
xlabel('Frequency (MHz)'), ylabel('Magnitude (dB)'), title('Spectrum near 10 GHz');
```


2) Testing of MATLAB program:

Now that the first iteration of our program has been finished, it is time to start testing it. We start by doing some initial tests, varying the ϕ_{Bias} , V_{π} , and gain terms in the MZM, to get a mathematical background of how the system behaves without a filter. Much of the work is detailed in the slides [here](#) but the main results are posted below.

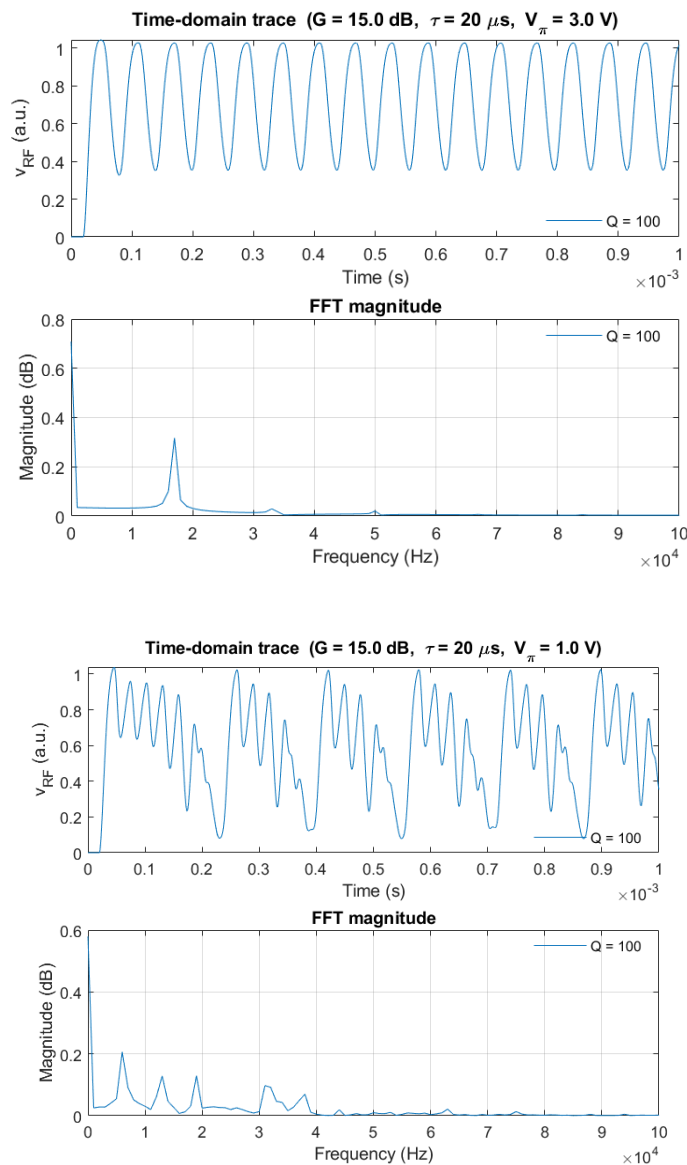
As seen in the appendix, the main program has been updated several times. The math, blocks used, and computational structure of the system itself remains the same. The main changes include changing the bandwidth so that the filter becomes relevant, getting rid of redundant uses of the carrier frequency and altering other parameters (such as the delay, τ_D). Our final iteration for this section and the one which produced the figures below is [Iteration 3](#).

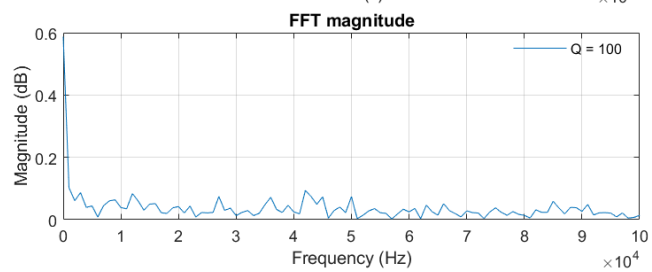
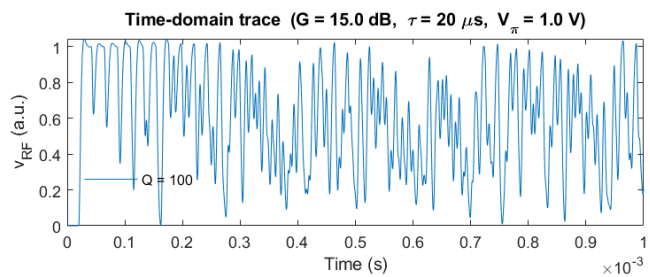
Based on our parameters, there are 4 specific shapes we can get:



Notice that as we decrease V_{π} , the half-wave voltage, our signal becomes more chaotic (the difference between the top right and bottom left graph is that half-wave voltage for the bottom left is $\frac{1}{3}$ that of the top right). The difference between the bottom left and bottom right graph is not explicitly shown, but resulted from the tripling of the bandwidth (BW) from the bottom left to bottom right graph. This makes sense as a lower value of the BW, for our case, would suppress high-frequency noise - something that is very much present in the bottom right graph.

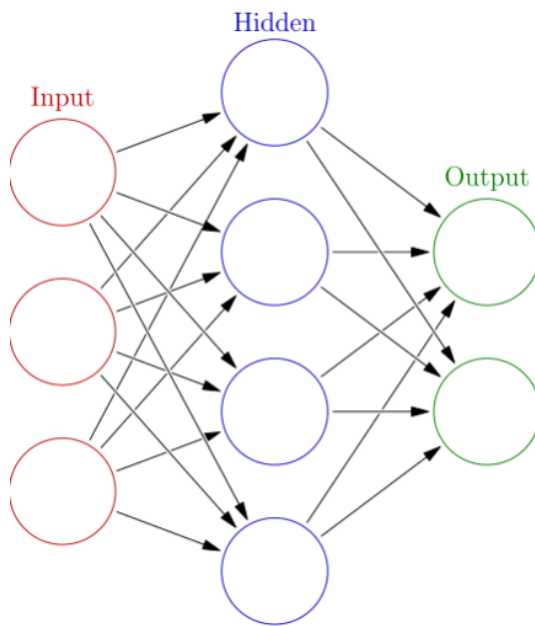
The frequency spectrum (used by applying the [Single Side FFT algorithm](#)) for the 2nd to 4th graphs is shown below:





3) Creation of Neural Network based on the OEO

The next step is to create a Neural Network based on the operation of the OEO. It would be helpful to first explain the structure of the Neural Network itself.



The first layer (or Input layer) is where the inputs of the system would go. For our case, this would be a signal entering the OEO. The final layer (or Output layer) is what gives the output of our Neural Network. In our case, it is a classification scheme that describes the different format of outputs. The hidden layer is a combination of values and weights that help correctly classify the different results of the system. We can use a training set (in which we already know the output beforehand) to tune the particular weights and values that give us the desired classification of our input.

If this seems a bit abstract, a more concrete example regarding flowers can be given. Imagine we have a dataset of flowers (but we do not know what the flower species are) and for each flower, attributes such as the number of petals, color, diameter, etc are given. The goal of the neural network is to classify the dataset of flowers such that the correct species of flower is predicted for each flower given its attributes. Let's assume that we have two datasets, Set 1 which only contains the attributes of the flowers and Set 2 which contains those attributes AND species of the flower.

We can start by inputting Set 2 into the system for which we can then update the values and weights of the hidden layer such that the number of flowers whose species are correctly guessed is maximized. We can then input Set 1 into our neural network to then predict the species of flower for each of the flowers.

Back to our OEO system. By training our neural network by inputting different inputs and corresponding outputs, we can then tune our values and weights of the hidden layer to then predict the output of the OEO system only given an input with a high degree of accuracy.

The code for our neural network is given below:

```
% — Parameters


---


Qrf      = 100;           % RF quality factor
Glin_dB   = 15;           % RF amplifier gain (dB)
Vpi       = 1;            % Modulator half-wave voltage (V)

u0_vals  = linspace(0, 9e-2, 10);

t_h       = linspace(20e-6, 220e-6, 100);
H         = zeros(numel(t_h), numel(u0_vals)); % 100 x 10

for k = 1:numel(u0_vals)
    [t_s, y] = OEO(Qrf, Glin_dB, Vpi, u0_vals(k)); % run oscillator
    H(:,k)    = interp1(t_s, y, t_h, 'linear').'; % 100 x 1
end

O = [1 1 0 0 0 0 0 0 0 0;
     0 0 1 1 1 0 0 0 0 0;
     0 0 0 0 0 1 1 1 0 0;
     0 0 0 0 0 0 0 0 1 1];
M = O*inv(H);
```

Appendix:

Relevant Papers:

- 1) *Optoelectronic oscillators with time-delayed feedback*
- 2) *Tutorial: Photonic neural networks in delay systems*
- 3) *Reservoir Computing Using Laser Networks*

Programs Used:

Single Side FFT:

```
function [P1,f] = SingleSideFFT(X,Y)
%UNTITLED6 Summary of this function goes here
% Detailed explanation goes here
    T = (X(2)-X(1));    % Sampling period
    Fs = 1/T;           % Sampling frequency

    L = length(X);      % Trace length
    L = L - mod(L,2);
    f = Fs/L*(0:(L/2)); % frequency vector

    P0 = fft(Y);
    % P2 = abs(P0/L);
    P2 = (P0/L);
    P1 = P2(1:L/2+1);
    P1(2:end-1) = 2*P1(2:end-1);
end
```

Iteration 1:

```
clear; rng default % deterministic noise seed
%% Core Parameters:

fc = 10e9; % RF freq [Hz]
tauD = 20e-6; % fibre delay 20 μs
Qrf = 3000; % RF band-pass Q
Glin = 1; % small-signal loop gain [dB]
Vpi = 4; % MZM half-wave voltage [V]
phiBias = pi/2; % quadrature bias for max linearity
Fs = 2e9; dt = 1/Fs; % envelope-level sample rate / step
Nt = round(1e-3/dt); % simulate 1 ms (>50× τD)
Ndelay = round(tauD/dt); % integer delay in samples
assert(Ndelay>10, 'Increase Fs or shorten τD');

%% Blocks:

% MZM intensity transfer, bias φb
mzm = @(v) cos(pi*v/(2*Vpi) + phiBias).^2;

% Soft-limiting RF amplifier (unit saturation, Glin dB small-signal)
A0 = 10^(Glin/20);
amp = @(x) (A0*x)/max(A0,abs(x)); % gentle bend into 1 V max

% Base-band equivalent of RF band-pass (2-pole Butterworth LPF)
BW = fc/Qrf; f3 = BW/2; wn = 2*pi*f3; % rad/s cutoff
[b,a] = butter(2,wn/(pi*Fs)); % digital LPF

%% Run -----
bufOpt = zeros(Ndelay,1); % optical-delay circular buffer
y = complex(zeros(Nt,1)); % RF output record
noise = (randn(Nt,1)+1i*randn(Nt,1))*1e-6;

% IIR filter state for per-sample processing
zi = zeros(max(numel(a),numel(b))-1,1);
v_prev = 0; % previous RF sample feeding the MZM

for n = 1:Nt
    % MZM (intensity transfer) -----
    v_mzm = mzm(real(v_prev)); % intensity ∝ cos2(...)
    % Fibre delay -----
    k = mod(n-1,Ndelay) + 1; % circular-buffer index
    v_del = bufOpt(k); % intensity after τD
    bufOpt(k) = v_mzm; % write current intensity
    % Photo-detector (linear) -----
    v_pd = v_del; % voltage ∝ delayed intensity
    % RF amplifier -----
    v_amp = amp(v_pd); % soft-limiting gain
    % Narrowband RF filter -----
    [v_filt, zi] = filter(b,a,v_amp,zi); % 2-pole Butterworth
    % Close the loop -----
    y(n) = v_filt; % record & seed with noise
end
```

```

        v_prev = y(n)+noise(n);                                % feeds next-step MZM
    end

    %% Visualise -----

    t = (0:Nt-1)*dt;
    Y = fftshift(fft(y));
    f = Fs*(-Nt/2:Nt/2-1)/Nt;
    figure;
    subplot(2,1,1), plot(t*1e3, real(y));
    xlabel('Time (ms)'), ylabel('v_{RF} (a.u.)'), title('OEO start-up & steady-state');
    subplot(2,1,2), plot(f/1e6, 20*log10(abs(Y)/max(abs(Y))));
    xlim([fc/1e6-100 fc/1e6+100]), grid on
    xlabel('Frequency (MHz)'), ylabel('Magnitude (dB)'), title('Spectrum near 10 GHz');

```


Iteration 2:

```
clear; rng default % deterministic noise seed
%% -----

% Core parameters (delay-line OEO, base-band complex-envelope model)
fc = 10e6; % RF centre frequency (Hz)
tauD = 20e-6; % fibre delay 20 µs
Qrf = 3000; % RF filter Q-factor
Glin_dB = 2; % small-signal gain (dB)
Vpi = 4; % MZM half-wave voltage (V)
phiBias = pi/4; % quadrature bias for linearity
Fs = 2e9; % complex-envelope sample-rate (Hz)
dt = 1/Fs;
simTime = 1e-3; % total simulated time (1 ms)
Nt = round(simTime/dt);
Ndelay = round(tauD/dt);
assert(Ndelay>10, 'Increase Fs or shorten tauD');

%% -----
% Block definitions
mzm = @(v) cos(pi*v/(2*Vpi) + phiBias).^2; % intensity
A0 = 10^(Glin_dB/20); % linear gain
amp = @(x) A0*x ./ max(1,abs(A0*x)); % 1 V sat
BW = fc/Qrf; f3 = BW/2; % 3 dB BW
[b,a] = butter(2, 2*f3/Fs); % digital LPF
%% -----

% Simulation
bufOpt = zeros(Ndelay,1); % fibre delay circular buffer
y = complex(zeros(Nt,1));
noise = randn(Nt,1)*1e-6; % real base-band noise
zi = zeros(max(numel(a),numel(b))-1,1);
v_prev = 0;

for n = 1:Nt
    v_mzm = mzm(real(v_prev)); % MZM
    k = mod(n-1,Ndelay)+1; % delay index
    v_del = bufOpt(k); % fibre delay
    bufOpt(k) = v_mzm;
    v_amp = amp(v_del); % RF amp
    [v_filt,zi] = filter(b,a,v_amp,zi); % narrowband
    y(n) = v_filt;
    v_prev = y(n) + noise(n); % close loop
End

%% -----
% Visualisation
t = (0:Nt-1)*dt;
Y = fftshift(fft(y));
f_bb = Fs*(-Nt/2:Nt/2-1)/Nt; % base-band axis
f_plot = f_bb + fc; % centre at 10 MHz
subplot(2,1,1)
```

```

plot(t*1e3, real(y)), xlabel('Time (ms)')
ylabel('v_{RF} (a.u.)', title('OEO start-up & steady-state'))
subplot(2,1,2)
plot(f_plot/1e6, 20*log10(abs(Y)/max(abs(Y))))
xlim([fc/1e6-100 fc/1e6+100]), grid on
xlabel('Frequency (MHz)'), ylabel('Magnitude (dB)')
title('Spectrum near 10 GHz')

```

Iteration 2 test (Multiple Q's):

```

% OEO_Qrf_sweep.m
% Delay-line optoelectronic oscillator - three Q values.
% Legend lists the varying Q; title lists the common parameters.
% -----

clear; rng default
% Sweep list and fixed parameters -----
Qvals = [50 150 300]; % RF filter Q-factors (varied)
Glin_dB = 2; % small-signal gain (dB) (fixed)
Vpi = 4; % MZM half-wave voltage (V) (fixed)
tauD = 20e-6; % fibre delay (s) (fixed in run_oeo)

% -----
figure('Color','w');
ax1 = subplot(1,1,1); hold(ax1,'on'); box(ax1,'on');

% --- plot traces, attach Q to each legend entry -----
for idx = 1:numel(Qvals)
    Qrf = Qvals(idx);
    [t_ms, y_rf] = run_oeo(Qrf, Glin_dB, Vpi);
    plot(ax1, t_ms, y_rf, ...
        'DisplayName', sprintf('Q = %d', Qrf));
end
% --- legend: now shows only the three Q values -----
legend(ax1,'Interpreter','tex','Location','best','Box','off');
% --- title with common parameters -----
title(ax1, sprintf('Time-domain traces (G = %.1f dB, \\\tau = %.0f \\\mus,
V_{\\pi} = %.1f V)', ...
    Glin_dB, tauD*1e6, Vpi), ...
    'Interpreter','tex');
xlabel(ax1,'Time (ms)');
ylabel(ax1,'v_{RF} (a.u.)');

% =====
% Core OEO model (unchanged) -----
function [t_ms,y_real,f_MHz,Y_dB] = run_oeo(Qrf,Glin_dB,Vpi)
    fc = 1e7; % RF centre frequency (Hz)
    tauD = 20e-6; % fibre delay (s)

```

```

phiBias = pi/4; % MZM bias
Fs = 2e9; % sample rate (Hz)
dt = 1/Fs;
simTime = 4e-4; % 0.4 ms total
Nt = round(simTime/dt);
Ndelay = round(tauD/dt);
% component models -----
mzm = @(v) cos(pi*v/(2*vpi)+phiBias).^2;
A0 = 10.^(Glin_dB/20);
amp = @(x) A0.*x./max(1,abs(A0.*x));
BW = fc/Qrf; % 3-dB bandwidth
[b,a] = butter(2,2*(BW/2)/Fs);
% simulation loop -----
buf = zeros(Ndelay,1);
y = complex(zeros(Nt,1));
noise = randn(Nt,1)*1e-6;
zi = zeros(max(numel(a),numel(b))-1,1);
v_prev = 0;
for n = 1:Nt
    v_mzm = mzm(real(v_prev));
    k = mod(n-1,Ndelay)+1;
    v_del = buf(k);
    buf(k) = v_mzm;
    v_amp = amp(v_del);
    [v_filt,zi] = filter(b,a,v_amp,zi);
    y(n) = v_filt;
    v_prev = y(n)+noise(n);
end
% derived outputs -----
t_ms = (0:Nt-1)*dt*1e3;
Y = fftshift(fft(y));
f_bb = Fs*(-Nt/2:Nt/2-1)/Nt;
f_MHz = (f_bb+fc)/1e6;
Y_dB = 20*log10(abs(Y)/max(abs(Y)));
y_real = real(y);
end

```

Iteration 2 test (Single Q-value):

```
% OEO_singleQ.m
% Delay-line optoelectronic oscillator - one Q value, no sweep loop
% -----

clear; rng default
% Fixed parameters -----
Qrf    = 100;      % RF filter Q-factor
Glin_dB = 15;      % small-signal gain (dB)
Vpi    = 1;        % MZM half-wave voltage (V)
tauD    = 20e-6;   % fibre delay (s) (run_oeo constant)

% -----
figure('Color','w');
ax1 = subplot(1,1,1); hold(ax1,'on'); box(ax1,'on');

% --- run model once and plot -----
[t_ms, y_rf] = run_oeo(Qrf, Glin_dB, Vpi);
plot(ax1, t_ms, y_rf, 'DisplayName', sprintf('Q = %d', Qrf));
legend(ax1, 'Interpreter', 'tex', 'Location', 'best', 'Box', 'off');
title(ax1, sprintf('Time-domain trace (G = %.1f dB, \\tau = %.0f \\mus, V_{\\pi} = %.1f V)', ...
    Glin_dB, tauD*1e6, Vpi), ...
    'Interpreter', 'tex');
xlabel(ax1, 'Time (ms)');
ylabel(ax1, 'v_{RF} (a.u.)');

% =====
% Core OEO model (unchanged) -----
function [t_ms, y_real, f_MHz, Y_dB] = run_oeo(Qrf, Glin_dB, Vpi)
    tauD    = 20e-6;      % fibre delay (s)
    phiBias = pi/4;       % MZM bias
    Fs      = 2e9;        % sample rate (Hz)
    dt      = 1/Fs;
    simTime = 1e-3;       % 1 ms total
    Nt      = round(simTime/dt);
    Ndelay  = round(tauD/dt);
    % component models -----
    mzm = @(v) cos(pi*v/(2*Vpi)+phiBias).^2;
    A0   = 10.^(Glin_dB/20);
    amp  = @(x) A0.*x./max(1,abs(A0.*x));
    BW   = 1/tauD;         % 3-dB bandwidth
    [b,a] = butter(2,2*(BW/2)/Fs);
    % simulation loop -----
    buf  = zeros(Ndelay,1);
    y     = complex(zeros(Nt,1));
    noise = randn(Nt,1)*1e-6;
    zi    = zeros(max(numel(a),numel(b))-1,1);
    v_prev = 0;
    for n = 1:Nt
        v_mzm = mzm(real(v_prev));
        k      = mod(n-1,Ndelay)+1;
```

```

        v_del      = buf(k);
        buf(k)     = v_mzm;
        v_amp      = amp(v_del);
        [v_filt,zi] = filter(b,a,v_amp,zi);
        y(n)       = v_filt;
        v_prev     = y(n)+noise(n);
    end
    % derived outputs -----
    t_ms = (0:Nt-1)*dt*1e3;
    Y     = fftshift(fft(y));
    f_bb  = Fs*(-Nt/2:Nt/2-1)/Nt;
    Y_dB  = 20*log10(abs(Y)/max(abs(Y)));
    y_real = real(y);
    f_MHz = (f_bb + 1/tauD) / 1e6;    %#-ok<NASGU>
end

```

Iteration 3 test:

```
clear; rng default

% Fixed parameters -----
Qrf      = 100;      % RF filter Q-factor
Glin_dB   = 15;      % small-signal gain (dB)
Vpi      = 1;       % MZM half-wave voltage (V)
tauD      = 20e-6;   % fibre delay (s)
% -----

%% ----- TIME-DOMAIN TRACE -----
figure('Color','w');
ax1 = subplot(2,1,1); hold(ax1,'on'); box(ax1,'on');
% run model once and plot
[t_s, y] = run_oeo(Qrf, Glin_dB, Vpi);
[Y, f] = SingleSideFFT(t_s, y);
plot(ax1, t_s, y, 'DisplayName', sprintf('Q = %d', Qrf));
legend(ax1, 'Interpreter', 'tex', 'Location', 'best', 'Box', 'off');
title(ax1, sprintf('Time-domain trace (G = %.1f dB, \\\tau = %.0f \\\mus, V_{\\pi} = %.1f V)', ...
                  Glin_dB, tauD*1e6, Vpi), 'Interpreter', 'tex');
xlabel(ax1, 'Time (s)');
ylabel(ax1, 'v_{RF} (a.u.)');

%% ----- FFT MAGNITUDE -----
ax2 = subplot(2,1,2); hold(ax2,'on'); box(ax2,'on');
plot(ax2, f, abs(Y), 'DisplayName', sprintf('Q = %d', Qrf));
xlim([0,1e5])
xlabel(ax2, 'Frequency (Hz)');
ylabel(ax2, 'Magnitude (dB)');
title(ax2, 'FFT magnitude');
grid(ax2, 'on');
legend(ax2, 'Location', 'best', 'Box', 'off');

% =====
% Core OEO model -----
function [t_s,y] = run_oeo(Qrf,Glin_dB,Vpi)
    tauD      = 20e-6;      % fibre delay (s)
    phiBias   = pi/4;      % MZM bias
    Fs        = 2e9;      % sample rate (Hz)
    dt        = 1/Fs;
    simTime   = 1e-3;      % 1 ms total
    Nt        = round(simTime/dt);
    Ndelay    = round(tauD/dt);

    % component models -----
    mzm = @(v) cos(pi*v/(2*Vpi)+phiBias).^2;
    A0   = 10.^(Glin_dB/20);
    amp  = @(x) A0.*x./max(1,abs(A0.*x));
```

```

BW    = 1/tauD;                                % 3-dB bandwidth
[b,a] = butter(2,2*(BW/2)/Fs);

% simulation loop -----
buf    = zeros(Ndelay,1);
y       = complex(zeros(Nt,1));
noise  = randn(Nt,1)*1e-6;
zi      = zeros(max(numel(a),numel(b))-1,1);
v_prev = 0;

for n = 1:Nt
    v_mzm      = mzm(real(v_prev));
    k          = mod(n-1,Ndelay)+1;
    v_del      = buf(k);
    buf(k)     = v_mzm;
    v_amp      = amp(v_del);
    [v_filt,zi] = filter(b,a,v_amp,zi);
    y(n)       = v_filt;
    v_prev     = y(n)+noise(n);
end
t_s      = (0:Nt-1)*dt;
end

```

Iteration 4 (for OEO Neural Network):

```
function [t_s,y] = OEO(Qrf,Glin_dB,Vpi,u0)
    rng default
    if nargin<4, u0 = 0; end % default: no injection
    tauD = 20e-6;
    phiBias = pi/4;
    Fs = 2e9;
    dt = 1/Fs;
    simTime = 1e-3;
    Nt = round(simTime/dt);
    Ndelay = round(tauD/dt);
    mzm = @(v) cos(pi*v/(2*Vpi)+phiBias).^2;
    A0 = 10.^(Glin_dB/20);
    amp = @(x) A0.*x./max(1,abs(A0.*x));
    BW = 3/tauD;
    [b,a] = butter(2,2*(BW/2)/Fs);
    buf = zeros(Ndelay,1);
    y = zeros(Nt,1);
    noise = randn(Nt,1)*1e-6;
    zi = zeros(max(numel(a),numel(b))-1,1);
    v_prev = 0;
    for n = 1:Nt
        v_mzm = mzm(v_prev); % optical modulation
        k = mod(n-1,Ndelay)+1;
        v_del = buf(k); % retrieve delayed sample
        buf(k) = v_mzm; % store new sample
        v_amp = amp(v_del); % RF amplifier
        [v_filt,zi] = filter(b,a,v_amp,zi); % RF filter
        y(n) = v_filt; % loop output
        v_prev = y(n) + noise(n) + u0; % ← constant injection
    end
    t_s = (0:Nt-1)*dt;
end
```


Neural Network Test:

```
% — Parameters

Qrf      = 100;          % RF quality factor
Glin_dB   = 15;          % RF amplifier gain (dB)
Vpi       = 1;           % Modulator half-wave voltage (V)

u0_vals  = linspace(0, 9e-2, 10);

t_h       = linspace(20e-6, 220e-6, 100);
H         = zeros(numel(t_h), numel(u0_vals)); % 100 x 10

for k = 1:numel(u0_vals)
    [t_s, y] = OEO(Qrf, Glin_dB, Vpi, u0_vals(k)); % run oscillator
    H(:,k)    = interp1(t_s, y, t_h, 'linear').'; % 100 x 1
end

O = [1 1 0 0 0 0 0 0 0 0;
     0 0 1 1 1 0 0 0 0 0;
     0 0 0 0 0 1 1 1 0 0;
     0 0 0 0 0 0 0 0 1 1];
M = O*inv(H);
```